

Chat Client Using Microsoft Foundry Model

A simple interactive chat client that talks to a model deployed in Azure AI Foundry (e.g. DeepSeek-V3.2) via the OpenAI-compatible inference endpoint.

The conversation history is kept locally and replayed on every turn, so the model sees the full context. Runs until you press Ctrl+C.

Config values: AZURE_FOUNDRY_ENDPOINT, AZURE_FOUNDRY_API_KEY, AZURE_FOUNDRY_MODEL, optional SYSTEM_PROMPT.

Requirements

OpenAI

python-dotenv>=1.0.0

chatclient.py

```
import os
import sys
from dotenv import load_dotenv
from openai import OpenAI

CONFIG_FILE = "config.env"

def load_config():
    if not os.path.exists(CONFIG_FILE):
        print(f"Error: Config file '{CONFIG_FILE}' not found.")
        print("Copy config.env.example to config.env and fill in your values.")
        sys.exit(1)
    load_dotenv(CONFIG_FILE)

def get_required(key: str) -> str:
    value = os.getenv(key)
    if not value:
        print(f"Error: '{key}' is not set in {CONFIG_FILE}")
        sys.exit(1)
    return value
```

```

def build_client() -> tuple[OpenAI, str]:
    endpoint = get_required("AZURE_FOUNDRY_ENDPOINT").rstrip("/")
    api_key = get_required("AZURE_FOUNDRY_API_KEY")
    model = get_required("AZURE_FOUNDRY_MODEL")
    client = OpenAI(
        base_url=f"{endpoint}",
        api_key=api_key,
    )
    return client, model

def main():
    load_config()
    client, model = build_client()

    system_prompt = os.getenv("SYSTEM_PROMPT", "You are a helpful assistant.")
    conversation = [{"role": "system", "content": system_prompt}]

    print(f"Chat client connected to model: {model}")
    print("Type your message and press Enter. Press Ctrl+C to exit.\n")

    while True:
        try:
            user_input = input("You: ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nGoodbye!")
            break

        if not user_input:
            continue

        conversation.append({"role": "user", "content": user_input})

```

```
try:
    response = client.chat.completions.create(
        model=model,
        messages=conversation,
    )
    assistant_message = response.choices[0].message.content
    conversation.append({"role": "assistant", "content": assistant_message})
    print(f"\nAssistant: {assistant_message}\n")
except Exception as e:
    print(f"\nError calling model: {e}\n")
    conversation.pop()

if __name__ == "__main__":
    main()
```

config.env.example

```
# Azure AI Foundry configuration
# Copy this file to config.env and fill in your values

# The endpoint URL for your Azure AI Foundry serverless deployment
# Example: https://DeepSeek-V3-2-<suffix>.<region>.models.ai.azure.com
AZURE_FOUNDRY_ENDPOINT=https://your-deployment.region.models.ai.azure.com

# Your Azure AI Foundry API key
AZURE_FOUNDRY_API_KEY=your-api-key-here

# The model deployment name to use
# Example: DeepSeek-V3-2, gpt-4o, Phi-4, Meta-Llama-3-8B-Instruct
AZURE_FOUNDRY_MODEL=your-model-deployment-name

# Optional: system prompt to set assistant behavior
SYSTEM_PROMPT=You are a helpful assistant.
```

Chat Client Using Microsoft Foundry Agent

Same interactive chat experience, but instead of calling a raw model it invokes a published Azure AI Foundry **agent** (with its instructions, tools, and personality baked in). Uses `azure-ai-projects` to resolve the agent by name + version, then the OpenAI Responses API with `agent_reference` and `previous_response_id` to maintain conversation state on the service side. Authenticates via `DefaultAzureCredential` (your `az login` session).

Config values: `AZURE_FOUNDRY_ENDPOINT` (project endpoint), `AZURE_FOUNDRY_AGENT_NAME`, `AZURE_FOUNDRY_AGENT_VERSION`.

Requirements

```
azure-ai-projects>=2.1.0
azure-identity
openai
python-dotenv>=1.0.0
```

chatclient.py

```
import os
import sys
from dotenv import load_dotenv

sys.stdout.reconfigure(encoding="utf-8")
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential

CONFIG_FILE = "config.env"

def load_config():
    if not os.path.exists(CONFIG_FILE):
        print(f"Error: Config file '{CONFIG_FILE}' not found.")
        print("Copy config.env.example to config.env and fill in your values.")
        sys.exit(1)
    load_dotenv(CONFIG_FILE)
```

```

def get_required(key: str) -> str:
    value = os.getenv(key)
    if not value:
        print(f"Error: '{key}' is not set in {CONFIG_FILE}")
        sys.exit(1)
    return value

def build_clients():
    endpoint = get_required("AZURE_FOUNDRY_ENDPOINT")
    agent_name = get_required("AZURE_FOUNDRY_AGENT_NAME")
    agent_version = get_required("AZURE_FOUNDRY_AGENT_VERSION")

    project_client = AIProjectClient(
        endpoint=endpoint,
        credential=DefaultAzureCredential(),
    )
    openai_client = project_client.get_openai_client()
    return openai_client, agent_name, agent_version

def main():
    load_config()
    openai_client, agent_name, agent_version = build_clients()

    print(f"Chat client connected to agent: {agent_name} v{agent_version}")
    print("Type your message and press Enter. Press Ctrl+C to exit.\n")

    agent_reference = {
        "agent_reference": {
            "name": agent_name,
            "version": agent_version,
            "type": "agent_reference",
        }
    }
    previous_response_id = None

```

```
while True:
    try:
        user_input = input("You: ").strip()
    except (KeyboardInterrupt, EOFError):
        print("\nGoodbye!")
        break

    if not user_input:
        continue

    try:
        kwargs = {
            "input": [{"role": "user", "content": user_input}],
            "extra_body": agent_reference,
        }
        if previous_response_id:
            kwargs["previous_response_id"] = previous_response_id

        response = openai_client.responses.create(**kwargs)
        previous_response_id = response.id

        print(f"\nAssistant: {response.output_text}\n")
    except Exception as e:
        print(f"\nError: {e}\n")

if __name__ == "__main__":
    main()
```

config.env.example

```
# Azure AI Foundry Agent configuration
# Copy this file to config.env and fill in your values

# Your Azure AI Foundry project endpoint
# Found under: your project > Settings > Project details > Endpoint
AZURE_FOUNDRY_ENDPOINT=https://your-project.api.azureml.ms

# Your Azure AI Foundry API key
AZURE_FOUNDRY_API_KEY=your_key

# The name of the agent as it appears in the Foundry portal
AZURE_FOUNDRY_AGENT_NAME=ai901agent

# The version label for display purposes
AZURE_FOUNDRY_AGENT_VERSION=v2

azure-ai-projects>=2.1.0
azure-identity
openai
python-dotenv>=1.0.0
```

Text Analytics: Summarization, NER, Key Phrases

Runs a sample article through the Azure AI Language service using the `azure-ai-textanalytics` SDK and prints three results: an extractive summary, the named entities it found (with category and confidence), and the extracted key phrases. The script (`textanalytics.py`) is a single linear flow with no interactive prompt — meant as a readable reference for students. Authenticates with an endpoint + key.

Config values: `AZURE_LANGUAGE_ENDPOINT`, `AZURE_LANGUAGE_KEY`.

Requirements

`azure-ai-textanalytics` ≥ 5.3.0

`python-dotenv` ≥ 1.0.0

`textanalytics.py`

```
import os
import sys
from dotenv import load_dotenv
from azure.core.credentials import AzureKeyCredential
from azure.core.exceptions import AzureError
from azure.ai.textanalytics import TextAnalyticsClient

CONFIG_FILE = "config.env"

# A sample article we will analyze. In a real app this text would come from
# a user, a file, a database, etc.
SAMPLE_TEXT = (
    "Contoso Travel launched its new mobile app in Seattle last March. "
    "The app helps travelers book flights, hotels, and rental cars in one place. "
    "CEO Maria Gonzalez said the company saw a forty percent increase in bookings "
    "within the first month. Customers praised the simple design and fast support. "
    "Contoso plans to expand the service to Europe and Asia by the end of the year."
)
```



```

def load_config():
    if not os.path.exists(CONFIG_FILE):
        print(f"Error: Config file '{CONFIG_FILE}' not found.")
        print("Copy config.env.example to config.env and fill in your values.")
        sys.exit(1)
    load_dotenv(CONFIG_FILE)

def get_required(key: str) -> str:
    value = os.getenv(key)
    if not value:
        print(f"Error: '{key}' is not set in {CONFIG_FILE}")
        sys.exit(1)
    return value

def build_client() -> TextAnalyticsClient:
    endpoint = get_required("AZURE_LANGUAGE_ENDPOINT").rstrip("/")
    key = get_required("AZURE_LANGUAGE_KEY")
    return TextAnalyticsClient(
        endpoint=endpoint,
        credential=AzureKeyCredential(key),
    )

def show_summary(client: TextAnalyticsClient, documents: list[str]):
    print("=== Extractive Summarization ===")
    # begin_extract_summary is a long-running operation: it returns a poller
    # that we wait on with .result().
    poller = client.begin_extract_summary(documents)
    for result in poller.result():
        if result.is_error:
            print(f"Error: {result.error.message}")
            continue
        # Sentences come back ranked; print them in their original order.
        for sentence in sorted(result.sentences, key=lambda s: s.offset):

```

```

        print(f"    - {sentence.text}")
    print()

def show_entities(client: TextAnalyticsClient, documents: list[str]):
    print("=== Named Entity Recognition ===")
    for result in client.recognize_entities(documents):
        if result.is_error:
            print(f"Error: {result.error.message}")
            continue
        for entity in result.entities:
            print(
                f"    - {entity.text} "
                f"({entity.category}, confidence {entity.confidence_score:.2f})"
            )
    print()

def show_key_phrases(client: TextAnalyticsClient, documents: list[str]):
    print("=== Key Phrase Extraction ===")
    for result in client.extract_key_phrases(documents):
        if result.is_error:
            print(f"Error: {result.error.message}")
            continue
        for phrase in result.key_phrases:
            print(f"    - {phrase}")
    print()

def main():
    load_config()
    client = build_client()

    documents = [SAMPLE_TEXT]

    print("Analyzing the following text:\n")

```

```
print(SAMPLE_TEXT)
print()

try:
    show_summary(client, documents)
    show_entities(client, documents)
    show_key_phrases(client, documents)
except AzureError as e:
    print(f"\nError calling the Azure AI Language service: {e}")
    print("Double-check AZURE_LANGUAGE_ENDPOINT and AZURE_LANGUAGE_KEY in config.env.")
    sys.exit(1)

if __name__ == "__main__":
    main()
```

config.env.example

```
# Azure AI Language (Text Analytics) configuration
# Copy this file to config.env and fill in your values

# The endpoint URL for your Azure AI Language resource
# Found in the Azure portal under your Language resource > Keys and Endpoint
# Example: https://your-resource.cognitiveservices.azure.com/
AZURE_LANGUAGE_ENDPOINT=https://your-resource.cognitiveservices.azure.com/

# API key for the Language resource
AZURE_LANGUAGE_KEY=your-language-key-here
```

Live Speech Transcription

Captures audio from your default microphone and streams it to the Azure AI Foundry Speech service, printing both interim partial results and finalized phrases to the console in real time. Press Enter to stop.

Config values: AZURE_SPEECH_ENDPOINT, AZURE_SPEECH_KEY.

Requirements

azure-cognitiveservices-speech

python-dotenv>=1.0.0

chatclient.py

```
import os
import sys
import azure.cognitiveservices.speech as speechsdk
from dotenv import load_dotenv

sys.stdout.reconfigure(encoding="utf-8")

CONFIG_FILE = "config.env"

def load_config():
    if not os.path.exists(CONFIG_FILE):
        print(f"Error: Config file '{CONFIG_FILE}' not found.")
        print("Copy config.env.example to config.env and fill in your values.")
        sys.exit(1)
    load_dotenv(CONFIG_FILE)

def get_required(key: str) -> str:
    value = os.getenv(key)
    if not value:
        print(f"Error: '{key}' is not set in {CONFIG_FILE}")
        sys.exit(1)
    return value
```

```

def main():
    load_config()
    endpoint = get_required("AZURE_SPEECH_ENDPOINT")
    speech_key = get_required("AZURE_SPEECH_KEY")

    speech_config = speechsdk.SpeechConfig(subscription=speech_key, endpoint=endpoint)
    audio_config = speechsdk.audio.AudioConfig(use_default_microphone=True)
    recognizer = speechsdk.SpeechRecognizer(
        speech_config=speech_config, audio_config=audio_config
    )

    def on_recognizing(evt):
        print(f"\r... {evt.result.text}", end="", flush=True)

    def on_recognized(evt):
        if evt.result.text:
            print(f"\r>>> {evt.result.text}")

    recognizer.recognizing.connect(on_recognizing)
    recognizer.recognized.connect(on_recognized)

    recognizer.start_continuous_recognition()
    print("Listening... speak into the microphone. Press Enter to stop.")
    try:
        input()
    except (KeyboardInterrupt, EOFError):
        pass
    finally:
        recognizer.stop_continuous_recognition()
        print("\nStopped.")

if __name__ == "__main__":
    main()

```

config.env.example

```
# Azure AI Foundry Speech configuration
# Copy this file to config.env and fill in your values

# Speech resource endpoint
# Found in the Foundry/Azure portal under your Speech resource > Keys and Endpoint
AZURE_SPEECH_ENDPOINT=https://your-resource.cognitiveservices.azure.com/

# API key for the Speech resource
AZURE_SPEECH_KEY=your-speech-key-here
```

Vision: Object Identification, Detection, and OCR

A lightweight vision demo using the `azure-ai-vision-imageanalysis` SDK. The script (`vision.py`) reads three local images and runs one capability on each: `example1.png` is captioned and tagged to identify its main object, `example2.png` is run through object detection to list every object with its bounding-box coordinates, and `example3.png` is OCR'd to print the text it contains. Supply your own images with those filenames.

Authenticates with an endpoint + key.

Config values: `AZURE_VISION_ENDPOINT`, `AZURE_VISION_KEY`.

Requirements

`azure-ai-vision-imageanalysis` >= 1.0.0

`python-dotenv` >= 1.0.0

vision.py

```
import os
import sys
from dotenv import load_dotenv
from azure.core.credentials import AzureKeyCredential
from azure.core.exceptions import AzureError
from azure.ai.vision.imageanalysis import ImageAnalysisClient
from azure.ai.vision.imageanalysis.models import VisualFeatures

CONFIG_FILE = "config.env"

# Image files we expect to find next to this script. Put your own pictures here with these names before
# running.
SINGLE_OBJECT_IMAGE = "example1.png"    # one main object to identify
MULTI_OBJECT_IMAGE = "example2.png"    # several objects to detect + locate
OCR_IMAGE = "example3.png"             # an image containing text to read

def load_config():
    if not os.path.exists(CONFIG_FILE):
        print(f"Error: Config file '{CONFIG_FILE}' not found.")
        print("Copy config.env.example to config.env and fill in your values.")
        sys.exit(1)
    load_dotenv(CONFIG_FILE)
```

```

def get_required(key: str) -> str:
    value = os.getenv(key)
    if not value:
        print(f"Error: '{key}' is not set in {CONFIG_FILE}")
        sys.exit(1)
    return value

def build_client() -> ImageAnalysisClient:
    endpoint = get_required("AZURE_VISION_ENDPOINT").rstrip("/")
    key = get_required("AZURE_VISION_KEY")
    return ImageAnalysisClient(
        endpoint=endpoint,
        credential=AzureKeyCredential(key),
    )

def read_image(path: str) -> bytes:
    if not os.path.exists(path):
        print(f"Error: image file '{path}' not found next to this script.")
        sys.exit(1)
    with open(path, "rb") as f:
        return f.read()

def identify_single_object(client: ImageAnalysisClient):
    print(f"=== Identifying the main object in '{SINGLE_OBJECT_IMAGE}' ===")
    image = read_image(SINGLE_OBJECT_IMAGE)
    result = client.analyze(
        image_data=image,
        visual_features=[VisualFeatures.CAPTION, VisualFeatures.TAGS],
    )

    if result.caption is not None:
        print(
            f"  It looks like: {result.caption.text} "
            f"(confidence {result.caption.confidence:.2f})"
        )

```



```
if result.tags is not None:
    print("  Top tags:")
    for tag in result.tags.list[:5]:
        print(f"    - {tag.name} (confidence {tag.confidence:.2f})")
print()
```

```
def detect_objects(client: ImageAnalysisClient):
    print(f"=== Detecting all objects in '{MULTI_OBJECT_IMAGE}' ===")
    image = read_image(MULTI_OBJECT_IMAGE)
    result = client.analyze(
        image_data=image,
        visual_features=[VisualFeatures.OBJECTS],
    )

    if result.objects is None or not result.objects.list:
        print("  No objects detected.")
        print()
        return

    for obj in result.objects.list:
        name = obj.tags[0].name if obj.tags else "object"
        confidence = obj.tags[0].confidence if obj.tags else 0.0
        box = obj.bounding_box
        print(
            f"    - {name} (confidence {confidence:.2f}) "
            f"at x={box.x}, y={box.y}, width={box.width}, height={box.height}"
        )
    print()
```

```

def extract_text(client: ImageAnalysisClient):
    print(f"=== Extracting text (OCR) from '{OCR_IMAGE}' ===")
    image = read_image(OCR_IMAGE)
    result = client.analyze(
        image_data=image,
        visual_features=[VisualFeatures.READ],
    )
    if result.read is None or not result.read.blocks:
        print("  No text found in the image.")
        print()
        return

    for block in result.read.blocks:
        for line in block.lines:
            print(f"  {line.text}")
    print()

def main():
    load_config()
    client = build_client()

    try:
        identify_single_object(client)
        detect_objects(client)
        extract_text(client)
    except AzureError as e:
        print(f"\nError calling the Azure AI Vision service: {e}")
        print("Double-check AZURE_VISION_ENDPOINT and AZURE_VISION_KEY in config.env.")
        sys.exit(1)

if __name__ == "__main__":
    main()

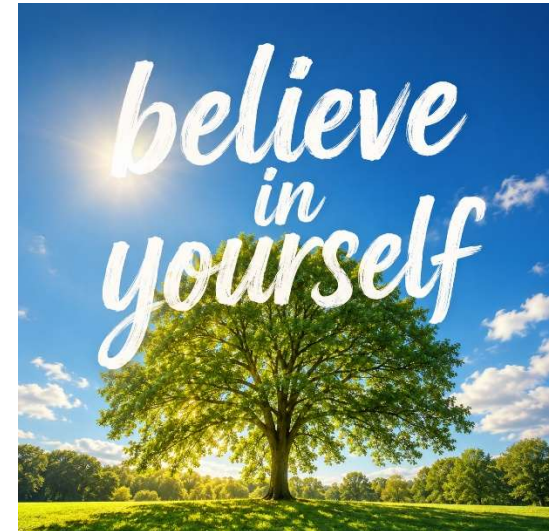
```

config.env.example

```
# Azure AI Vision configuration
# Copy this file to config.env and fill in your values

# The endpoint URL for your Azure AI Vision (Computer Vision) resource
# Found in the Azure portal under your Vision resource > Keys and Endpoint
# Example: https://your-resource.cognitiveservices.azure.com/
AZURE_VISION_ENDPOINT=https://your-resource.cognitiveservices.azure.com/

# API key for the Vision resource
AZURE_VISION_KEY=your-vision-key-here
```



Content Understanding: Information Extraction

A lightweight information-extraction demo built on the `azure-ai-contentunderstanding` SDK (GA, service API version 2025-11-01). The script (`content_understanding.py`) sends three local files to prebuilt analyzers via `begin_analyze_binary`: `example1.pdf` and `example2.jpg` (both invoices) go through `prebuilt-invoice` and their extracted fields are printed, and `example3.mp3` (a recorded course lesson) goes through `prebuilt-callCenter` — it listens to the recording and prints a concise summary of what was discussed. The SDK handles the long-running submit-and-poll operation. Authenticates with an endpoint + key.

Config values: `AZURE_CONTENT_UNDERSTANDING_ENDPOINT`, `AZURE_CONTENT_UNDERSTANDING_KEY`.

Requirements

`azure-ai-contentunderstanding` >= 1.1.0

`python-dotenv` >= 1.0.0

`contentunderstanding.py`

```
import os
import sys
from dotenv import load_dotenv
from azure.core.credentials import AzureKeyCredential
from azure.core.exceptions import AzureError
from azure.ai.contentunderstanding import ContentUnderstandingClient
from azure.ai.contentunderstanding.models import AudioVisualContent

CONFIG_FILE = "config.env"

# Files we expect to find next to this script. Drop your own samples in with
# these names before running.
INVOICE_PDF = "example1.pdf"    # an invoice as a PDF document
INVOICE_IMAGE = "example2.jpg"  # an invoice as an image
AUDIO_FILE = "example3.mp3"     # a recorded course lesson

# Content Understanding GA API version.
API_VERSION = "2025-11-01"
```

```

# Prebuilt analyzers provided by the service (no training required).
# prebuilt-invoice extracts structured invoice fields from PDFs *and* images.
# prebuilt-callCenter listens to recorded speech and produces a concise
# summary of what was discussed -- this is the analyzer Microsoft's own
# samples use for audio.
INVOICE_ANALYZER = "prebuilt-invoice"
AUDIO_ANALYZER = "prebuilt-callCenter"

def load_config():
    if not os.path.exists(CONFIG_FILE):
        print(f"Error: Config file '{CONFIG_FILE}' not found.")
        print("Copy config.env.example to config.env and fill in your values.")
        sys.exit(1)
    load_dotenv(CONFIG_FILE)

def get_required(key: str) -> str:
    value = os.getenv(key)
    if not value:
        print(f"Error: '{key}' is not set in {CONFIG_FILE}")
        sys.exit(1)
    return value

def build_client() -> ContentUnderstandingClient:
    endpoint = get_required("AZURE_CONTENT_UNDERSTANDING_ENDPOINT").rstrip("/")
    key = get_required("AZURE_CONTENT_UNDERSTANDING_KEY")
    return ContentUnderstandingClient(
        endpoint=endpoint,
        credential=AzureKeyCredential(key),
        api_version=API_VERSION,
    )

```

```

def read_file(path: str) -> bytes:
    if not os.path.exists(path):
        print(f"Error: file '{path}' not found next to this script.")
        sys.exit(1)
    with open(path, "rb") as f:
        return f.read()

def analyze_file(client: ContentUnderstandingClient, analyzer_id: str, path: str):
    """Send a local file to a prebuilt analyzer and wait for the result.

    Content Understanding analysis is a long-running operation. The SDK's
    begin_analyze_binary() returns a poller; calling .result() blocks
    until the service is done and hands back an AnalysisResult.
    """
    file_bytes = read_file(path)
    poller = client.begin_analyze_binary(analyzer_id, binary_input=file_bytes)
    return poller.result()

def print_invoice_fields(label: str, result):
    print(f"=== Extracted invoice data from '{label}' ===")
    if not result.contents:
        print("  No content returned.")
        print()
        return

    fields = result.contents[0].fields or {}
    if not fields:
        print("  No fields extracted.")
        print()
        return

    for name, field in fields.items():
        # Every field exposes a generic .value regardless of its type

```

```

        # (string, number, date, ...).
        if field.value not in (None, "", [], {}):
            print(f" {name}: {field.value}")
    print()

def get_audio_content(result):
    for content in result.contents or []:
        # The model is dict-like; match on the 'kind' discriminator rather
        # than isinstance.
        if isinstance(content, AudioVisualContent):
            return content
        if str(content.get("kind", "")) == "audioVisual":
            return content
    return None

def print_audio_summary(audio):
    fields = audio.fields or {}
    summary = fields.get("Summary")
    if summary and summary.value:
        print(f" Summary: {summary.value}")
    else:
        print(" No summary returned.")

def main():
    load_config()
    client = build_client()

    try:
        # 1. Invoice as a PDF document.
        pdf_result = analyze_file(client, INVOICE_ANALYZER, INVOICE_PDF)
        print_invoice_fields(INVOICE_PDF, pdf_result)

```

```

# 2. Invoice as an image.
img_result = analyze_file(client, INVOICE_ANALYZER, INVOICE_IMAGE)
print_invoice_fields(INVOICE_IMAGE, img_result)

# 3. Recorded course lesson -> spoken-content summary.
print(f"=== Summarizing audio '{AUDIO_FILE}' ===")
audio_result = analyze_file(client, AUDIO_ANALYZER, AUDIO_FILE)
audio = get_audio_content(audio_result)
if audio is None:
    print(" No audio content returned.")
else:
    print_audio_summary(audio)
print()
except AzureError as e:
    print(f"\nError calling Azure AI Content Understanding: {e}")
    print("Check AZURE_CONTENT_UNDERSTANDING_ENDPOINT / _KEY in config.env.")
    sys.exit(1)

if __name__ == "__main__":
    main()

```

config.env.example

```

# Azure AI Content Understanding configuration
# Copy this file to config.env and fill in your values

# The endpoint URL for your Azure AI Foundry / AI Services resource that has
# Content Understanding enabled.
# Found in the Azure portal under your resource > Keys and Endpoint
# Example: https://your-resource.cognitiveservices.azure.com/
AZURE_CONTENT_UNDERSTANDING_ENDPOINT=https://your-resource.cognitiveservices.azure.com/

# API key for the resource
AZURE_CONTENT_UNDERSTANDING_KEY=your-content-understanding-key-here

```